

# Testing complex agents

Where I would start on CI/CD, the golden set, the eval harness, and the one orchestration decision I think we should make deliberately rather than drift into.

Three positions: the golden set is the first artefact and it is not ours to own; what we make repeatable is the harness rather than the agent; and one orchestration standard per lane. The failure modes named below are ones I have watched happen on tools I built.

## Two kinds of test, kept apart

| WHAT                            | HOW IT IS TESTED                                                                                                                                                          | COST  |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| The interlocks around the model | Ordinary unit tests. Permission check, citation-range check, confidentiality check, approval gate — code with a deterministic answer, testable with no model in the room. | cheap |
| Every point the model judges    | A case in the golden set, run repeatedly, gated on a pass rate.                                                                                                           | not   |

Conflating them is how a team ends up with a green pipeline and an agent nobody trusts. The first is where most of the actual guarantees live. The second is where the risk lives.

## Where each level runs

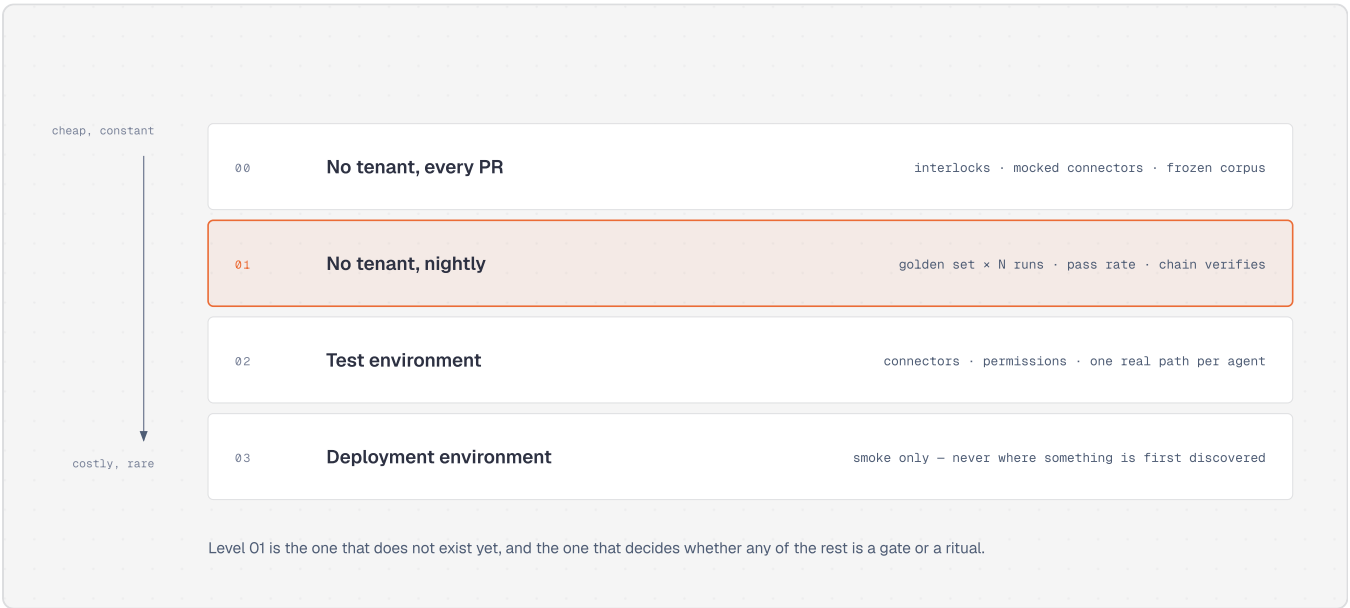


Fig. 1 — the cheap levels run constantly; the expensive ones prove integration, not behaviour.

The level worth insisting on is **00**: most tests should need no tenant, no licence and no network, because that is the only version that can run on every pull request.

If tests require a Dataverse environment to execute, they will run weekly and we will hear about regressions from users.

## The golden set

| SET                   | ANSWERS                                                    | WHY SEPARATE                                                                  |
|-----------------------|------------------------------------------------------------|-------------------------------------------------------------------------------|
| Retrieval quality     | Did we put the right documents in front of the model?      | An agent can be right for the wrong reason. We need to know which half broke. |
| Task correctness      | Given the right evidence, is the judgement right?          | The obvious one, and useless on its own.                                      |
| Refusal and negatives | What must it decline, and what must it say it cannot find? | The set teams skip, and where the exposure sits.                              |

Two live examples of the third mattering: a business-continuity control with no evidence in the pack at all, where a larger model reliably says *insufficient evidence* and a smaller one accepts an adjacent document; and a proposal brief with no comparable past engagement, where the correct output is a no-go rather than a confident price.

Production failures go back into the set — that is the difference between a gate and a snapshot. And the **subject-matter people own the cases**. If engineering owns them the set rots; we are not the people who can say what the right answer is on a control conclusion. Engineering owns the harness. Worth settling in the team's first month rather than at the first incident.

## The harness

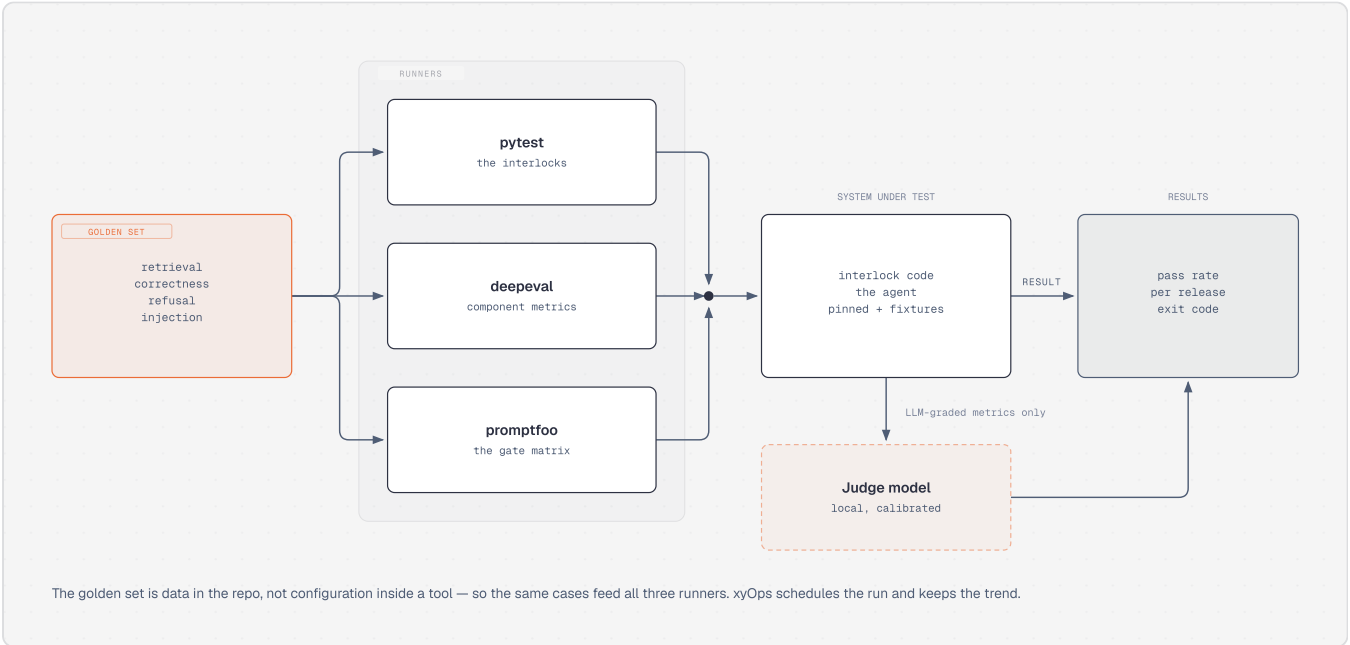


Fig. 2 – one golden set as data in the repo, three runners, one judge.

## promptfoo and deepeval, mapped

They are complementary, not competing. Both open source, both able to run against a local model, so level 00 works with no tenant.

| WHAT WE NEED TO KNOW                                 | TOOL      | HOW                                                   |
|------------------------------------------------------|-----------|-------------------------------------------------------|
| The interlocks behave                                | pytest    | plain asserts, no model involved                      |
| Retrieval quality, separate from the answer          | deepeval  | contextual precision, recall, relevancy               |
| The answer is supported by what was retrieved        | deepeval  | faithfulness, hallucination                           |
| The right tool was called, in the right order        | deepeval  | tool correctness, task completion                     |
| Domain judgement — "is this a defensible conclusion" | deepeval  | G-Eval against a written rubric                       |
| The gate: cases × repeats × pass rate, one exit code | promptfoo | YAML suite, assertions, provider matrix, CI exit code |
| Refusal cases                                        | promptfoo | not-contains and rubric assertions                    |
| Cost and latency budgets                             | promptfoo | per-case thresholds                                   |
| Injection and jailbreak resistance                   | promptfoo | red-team suites                                       |
| The audit chain verified as part of the pass         | ours      | a custom assertion in both                            |

### Three things to decide before adopting either.

- **The judge model is itself unmeasured.** Every graded metric needs one, and on the sovereign lane it has to be a local model. Calibrate it against human labels on a subset, or we have replaced one unmeasured model with two.
- **Keep the golden set as data in the repo** — YAML or JSON we own — not as configuration inside either tool. That is what lets the same cases feed all three runners and survive a tool change.
- **Neither replaces the interlock tests or the chain verification.** Those stay plain pytest.

## Repeatability: the harness, not the agent

There is no "deterministic agent". What we make deterministic is the harness around it.

| CONTROL           | DETAIL                                                                                                          |
|-------------------|-----------------------------------------------------------------------------------------------------------------|
| Pinned model      | One tag per run, expected digest in config, verified at startup. A mismatch should refuse, not warn.            |
| Temperature zero  | And hidden reasoning disabled where the model emits it — cost without benefit for structured judgement.         |
| Seeded selection  | Anything that samples derives its seed from the inputs, so a run reproduces and a reviewer can verify the draw. |
| Recorded fixtures | Tool and connector responses replayed, not called live.                                                         |
| Frozen corpus     | Versioned with the test suite.                                                                                  |

All five are already in the tools I have built — it is what made them demonstrable.

## Flakiness is data, not noise

A case runs N times and gates on a **pass rate**, not pass or fail. Two measurements make the argument better than theory: a comparable-selection step returned a different set of past engagements between two runs of the same brief, moving the median; and on a twelve-item sample, an attribute judgement flagged four of eight clean items as exceptions on one run and none on the next.

A single-run gate on that goes red or green at random, and the team learns to re-run until green — worse than no gate, because it teaches people to ignore it. A rate also catches what a binary gate cannot see at all: a case dropping from ten out of ten to seven out of ten has regressed while still "passing".

Record the rate, not the verdict. The trend across releases is the signal; any single run is close to meaningless.

## Tooling, described honestly

| TOOL                                          | WHAT IT ACTUALLY DOES                                                                                           |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Vibe Kanban<br>github.com/BloopAI/vibe-kanban | Gives each agent its own git worktree so several work in parallel without colliding. A <i>development</i> tool. |
| xyOps<br>github.com/pixlcore/xyops            | Schedules the checks, stores results, alerts. The CI/CD and monitoring plane.                                   |
| promptfoo · deepeval                          | The runners above. They evaluate; they do not schedule or isolate.                                              |
| The golden set                                | Ours. No tool supplies it.                                                                                      |

Neither Vibe Kanban nor xyOps is enterprise-supported. If we standardise on them that should be a deliberate decision with a short ADR behind it, not something we drift into because they were what one person already had running.

## Orchestration: one standard per lane

Not every agent needs a graph runtime such as LangGraph, and the criterion is not how often it is deployed. It is whether the workflow needs explicit, inspectable, **resumable state**.

| REACH FOR IT<br>WHEN                                               | DO NOT WHEN                                                                         |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| The flow branches on intermediate results                          | Single-turn retrieval and answer                                                    |
| A human approval pauses a run for hours or days and it must resume | A fixed pipeline with no branching — a plain function is smaller and easier to test |
| Partial failure needs to resume mid-flow, not from the top         | Anything Copilot Studio topics and Power Automate already express                   |
| Someone needs to see the state at each step                        | The state fits in one call and nobody will ask about it                             |

In a Microsoft estate this is a **stack decision, not a library choice** — a second deployment path, a second audit surface, a second set of skills to hire and review for. Decided once, at team level, not per agent by whoever is building it. What I would propose: the Microsoft path for the tenant-integrated lane, a graph runtime considered only for the sovereign lane where we control the whole stack. I already have four orchestration patterns running in Python with no third-party dependencies, so we can test that question against something real.

### What I would do first

- Pick one agent we already understand and build its golden set end to end, refusal cases included. It will be more work than anyone expects, and that is the useful part.
- Get level 00 running with no tenant, on every pull request.
- Agree who owns the cases.
- Write the orchestration ADR before the second agent, not after the fifth.